NSE Market Data Downloader & Live Analytics Suite

Complete Technical Architecture, Codebase Walkthrough & Operating Guide

Language: Python 3.14 **Stack:** curl_cffi, pandas, FastAPI, Vanilla HTML/CSS/JS **Author:** Engineering Team **Generated:** October 2026

1. Executive Summary & Objective

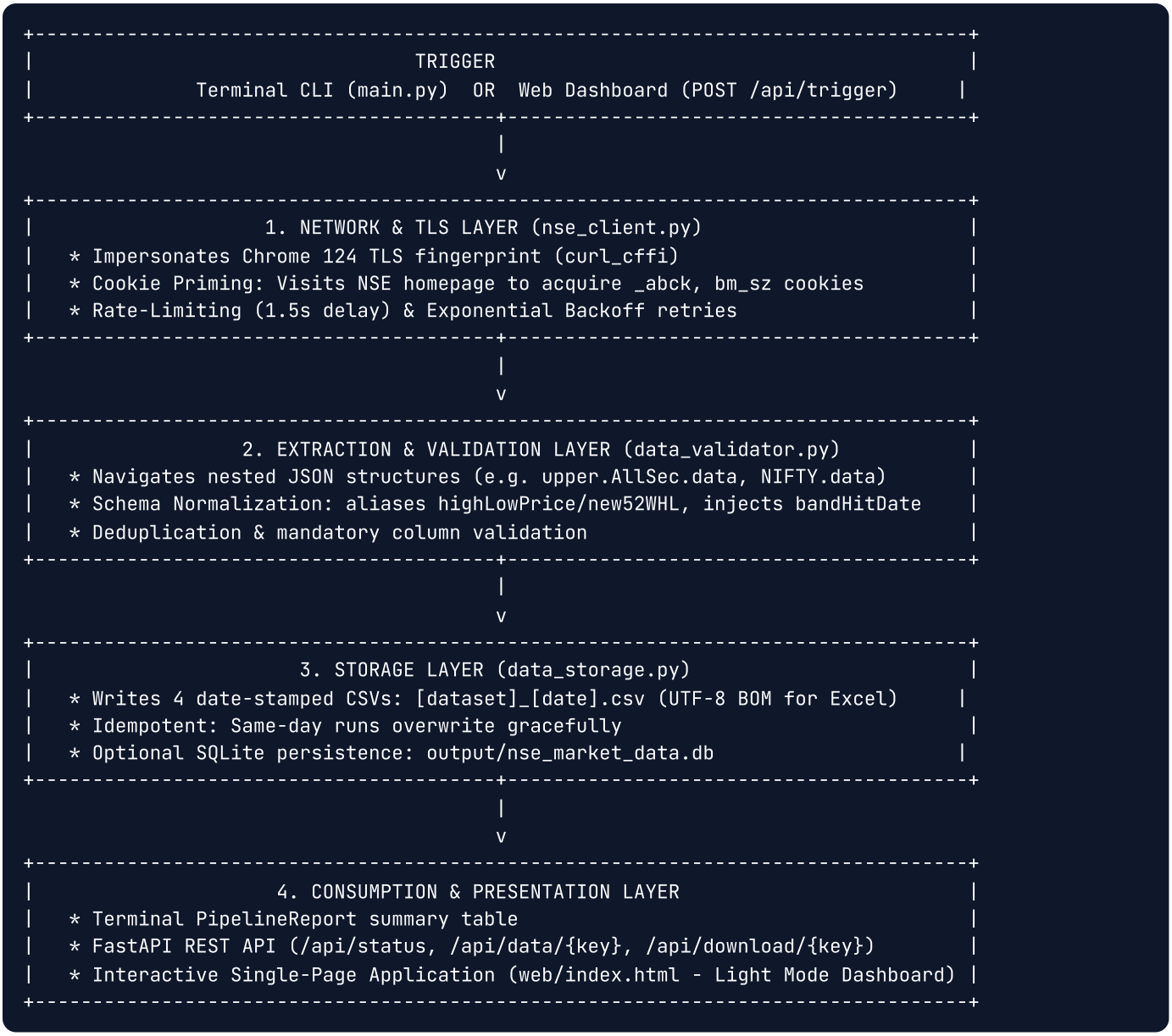
The **NSE Market Data Downloader** is a robust data engineering and analytics application. Its core mandate is to automatically retrieve live trading analytics from four key National Stock Exchange of India (NSE) market-data sections and export them into four clean, date-stamped CSV files:

- **Top Gainers & Losers:** `https://www.nseindia.com/market-data/top-gainers-losers`
- **Upper Band Hitters:** `https://www.nseindia.com/market-data/upper-band-hitters`
- **Volume Gainers / Spurts:** `https://www.nseindia.com/market-data/volume-gainers-spurts`
- **52-Week High — Equity Market:** `https://www.nseindia.com/market-data/52-week-high-equity-market`

> **The Akamai WAF & TLS Challenge**
>
> Standard scraping attempts with `urllib` or `requests` receive an immediate **HTTP 403 Forbidden**. NSE protects its infrastructure using Akamai Bot Manager, which performs TLS fingerprint inspection (JA3/JA4 ciphers), cookie validation, and Referer checking. This project overcomes these defenses using browser-level TLS impersonation with `curl_cffi` and cookie priming.

2. System Architecture & High-Level Data Flow

The system follows a modular ETL (Extract-Transform-Load) design pattern:


TRIGGER
Terminal CLI (main.py) OR Web Dashboard (POST /api/trigger)
1. NETWORK & TLS LAYER (nse_client.py)
* Impersonates Chrome 124 TLS fingerprint (curl_cffi)
* Cookie Priming: Visits NSE homepage to acquire _abck, bm_sz cookies
* Rate-Limiting (1.5s delay) & Exponential Backoff retries
2. EXTRACTION & VALIDATION LAYER (data_validator.py)
* Navigates nested JSON structures (e.g. upper.AllSec.data, NIFTY.data)
* Schema Normalization: aliases highLowPrice/new52WHL, injects bandHitDate
* Deduplication & mandatory column validation
3. STORAGE LAYER (data_storage.py)
* Writes 4 date-stamped CSVs: [dataset]_[date].csv (UTF-8 BOM for Excel)
* Idempotent: Same-day runs overwrite gracefully
* Optional SQLite persistence: output/nse_market_data.db
4. CONSUMPTION & PRESENTATION LAYER
* Terminal PipelineReport summary table
* FastAPI REST API (/api/status, /api/data/{key}, /api/download/{key})
* Interactive Single-Page Application (web/index.html - Light Mode Dashboard)

3. Detailed File-by-File Code Breakdown

1. config.py

`Configuration Layer`

Responsibility: Separates all endpoint paths, browser headers, timeouts, and tunable constants from the application code.

Key Components:

- `DatasetConfig` : Dataclass defining name, key, API endpoint, page URL, primary JSON key, and expected columns.
- `DATASETS` : Dictionary mapping the 4 targets to their exact internal NSE APIs:
  - `top-gainers-losers` → `/api/live-analysis-variations?index=gainers`
  - `upper-band-hitters` → `/api/live-analysis-price-band-hitter`
  - `volume-gainers-spurts` → `/api/live-analysis-volume-gainers`
  - `52-week-high` → `/api/live-analysis-data-52weekhighstock`
- `DEFAULT_HEADERS` : Emulates real Chrome browser headers (User-Agent, Accept-Encoding, Connection).
- `HTTPConfig` & `StorageConfig` : Configures 15s timeout, 3 retries, 1.5s rate limit, and `output/` directory.

2. nse_client.py

`Network & Security Layer`

Responsibility: Manages HTTP sessions, browser TLS fingerprinting, cookie acquisition, rate limiting, and network retries.

Key Code Logic:

```
# 1. TLS Impersonation via curl_cffi
def _create_session(self):
    if _USE_CURL_CFFI:
        return CurlSession(impersonate="chrome124")
    return requests.Session()

# 2. Cookie Priming
def initialise_session(self):
    self._session = self._create_session()
    self._session.headers.update(DEFAULT_HEADERS)
    resp = self._session.get("https://www.nseindia.com/", timeout=10)
    # Automatically collects _abck, ak_bmsc, bm_sz cookies

# 3. Transparent Retries with Exponential Backoff
for attempt in range(1, max_retries + 1):
    resp = self._session.get(url, headers={"Referer": referer})
    if resp.status_code in (401, 403):
        self.initialise_session()  # session expired: re-prime and retry
        continue
    return resp.json()
```

3. data_validator.py

`Extraction & Transformation Layer`

Responsibility: Extracts row dictionaries from nested JSON structures, normalizes schemas, verifies mandatory columns, and eliminates duplicate rows.

Key Code Logic:

```
# Multi-tier JSON Extraction
if dataset.key == "top-gainers-losers":
    return raw_json["NIFTY"]["data"]
elif dataset.key == "upper-band-hitters":
    return raw_json["upper"]["AllSec"]["data"]  # Extracts all 134 records!
elif dataset.key in ("volume-gainers-spurts", "52-week-high"):
    return raw_json["data"]

# Schema Normalization
if dataset.key == "52-week-high":
    if "highLowPrice" not in df.columns and "new52WHL" in df.columns:
        df["highLowPrice"] = df["new52WHL"]
elif dataset.key == "upper-band-hitters":
    if "bandHitDate" not in df.columns:
        df["bandHitDate"] = datetime.now().strftime("%d-%b-%Y")
```

4. data_storage.py

`Persistence Layer`

Responsibility: Persists validated DataFrames into date-stamped CSV files and SQLite tables.

Key Code Logic:

- `CSVStorage.save()` : Writes CSV using `encoding="utf-8-sig"` (Byte Order Mark), ensuring Excel correctly displays Indian Rupee symbols (₹) and accents.
- **Same-Day Idempotency:** Re-executing the script on the same trading day safely overwrites the day's CSV instead of creating duplicated files.
- `SQLiteStorage.save()` : Optional SQLite database writer (`output/nse_market_data.db`) with automatic `download_date` column stamping for multi-day historical analysis.

5. pipeline.py

`Pipeline Orchestrator`

Responsibility: Glues client, validator, and storage into an end-to-end fault-tolerant workflow.

Key Code Logic:

- `Pipeline.run(dataset_keys, date_str)` : Executes session priming once, then iterates through all target datasets.
- **Fault Isolation:** Each dataset download is isolated in a try-catch block; if one dataset experiences network failure, the remaining datasets continue processing.
- `PipelineReport` : Compiles execution times, row counts, and error messages into a clean summary report.

6. main.py

`Command-Line Interface`

Responsibility: User-facing CLI entry point with argument parsing and terminal encoding safety.

Key Features:

- Configures `sys.stdout` to UTF-8 with character replacement so Windows Command Prompt never crashes on unicode characters (✓, ✗, ₹).
- Supports arguments: `--dataset` , `--output-dir` , `--sqlite` , `--ui` , `--retries` , `--timeout` , `--verbose` , and `--list` .

7. web_app.py & web/ (index.html, style.css, app.js)

`Web Application & Dashboard`

Responsibility: Fast, modern financial web dashboard and REST API server.

Key Endpoints:

- `GET /api/status` : Real-time sync status, record counts, and timestamps.
- `GET /api/data/{key}` : Returns parsed JSON records for the active dataset.
- `GET /api/download/{key}` : Directly downloads the generated CSV file.
- `POST /api/trigger` : Initiates a live market data re-scrape and updates files.

4. Web Dashboard Showcase (Institutional Light Mode)

The web dashboard provides an interactive visual interface designed with a clean, institutional light-mode aesthetic:

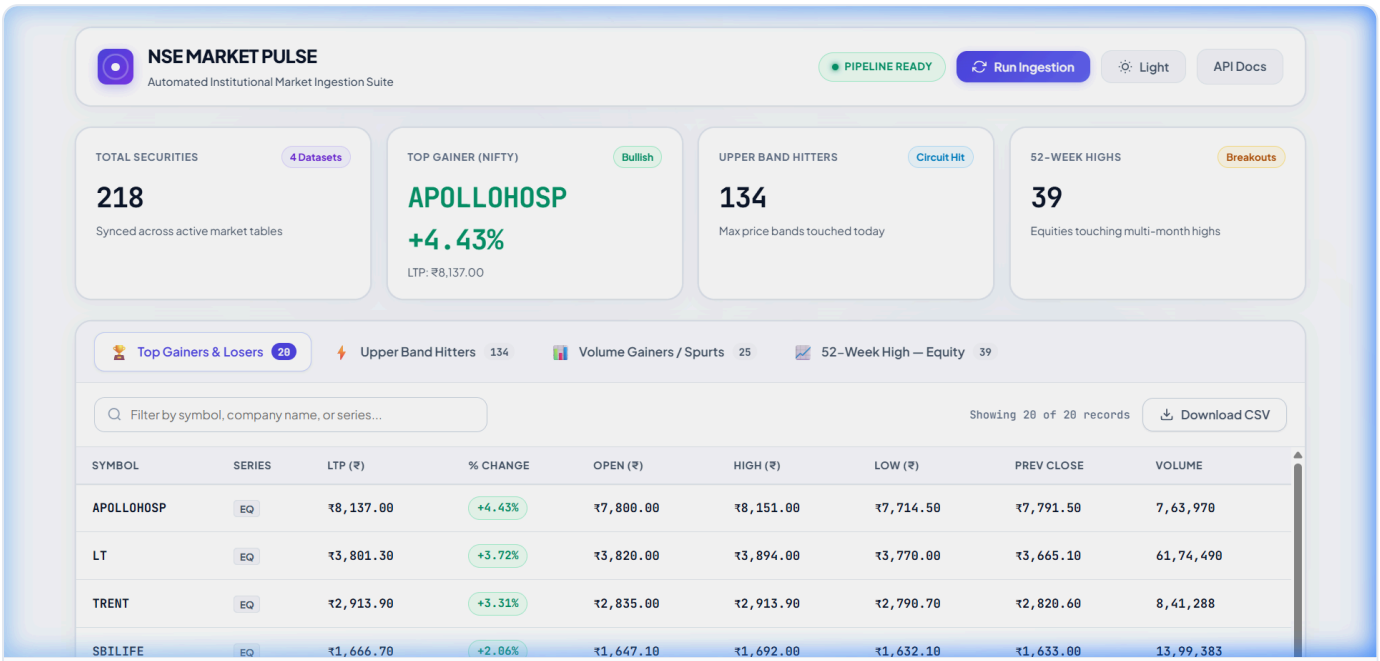


Figure 1: Main Dashboard in Light Mode displaying Top Gainers & Losers, live KPI metrics ribbon, search toolbar, and action controls.

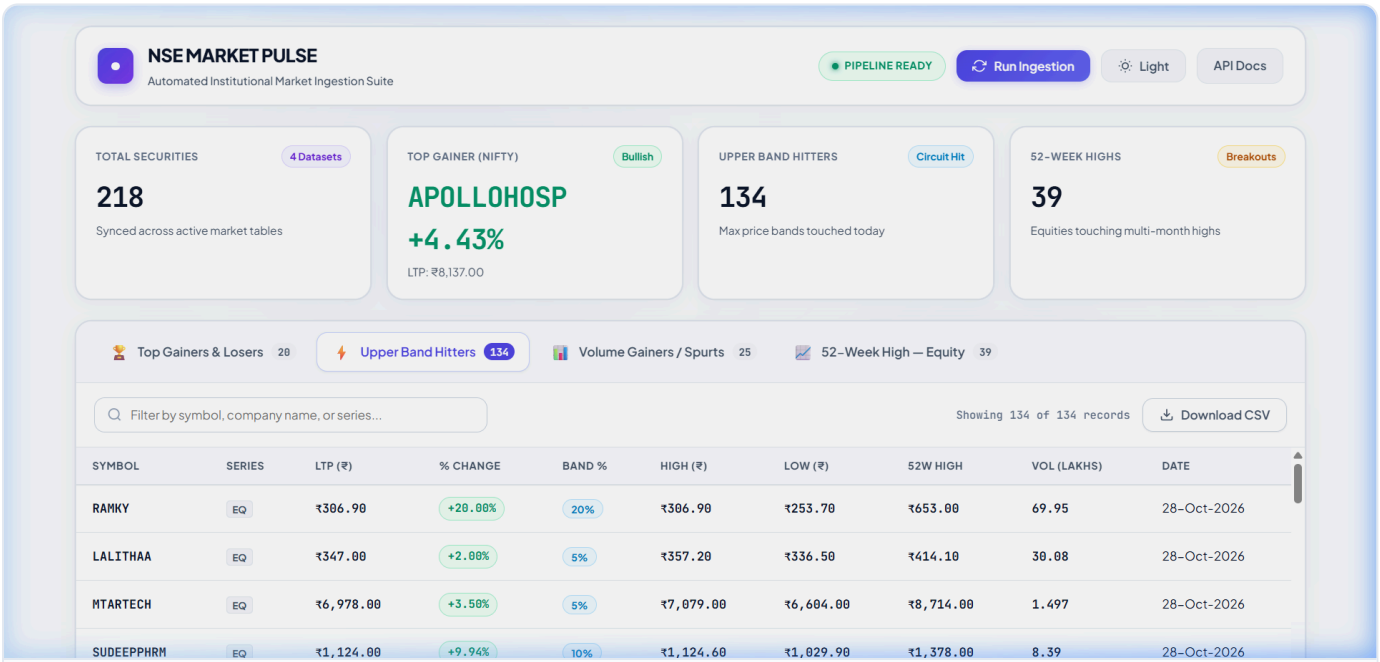


Figure 2: Upper Band Hitters view displaying all 134 securities, price band percentages (20%, 5%), and direct CSV download action.

5. Generated CSV Datasets Summary

DATASET NAME	OUTPUT CSV FILENAME	RECORDS	KEY COLUMNS EXTRACTED
Top Gainers & Losers	`top-gainers-losers_YYYY-MM-DD.csv`	20	symbol, series, open_price, high_price, low_price, ltp, prev_price, net_price, perChange, trade_quantity, turnover
Upper Band Hitters	`upper-band-hitters_YYYY-MM-DD.csv`	134	symbol, series, ltp, change, pChange, priceBand, highPrice, lowPrice, yearHigh, yearLow, totalTradedVol, turnover, bandHitDate
Volume Gainers / Spurts	`volume-gainers-spurts_YYYY-MM-DD.csv`	25	symbol, companyName, volume, week1AvgVolume, week1volChange, week2AvgVolume, week2volChange, ltp, pChange, turnover
52-Week High — Equity	`52-week-high_YYYY-MM-DD.csv`	39	symbol, series, comapnyName, ltp, change, pChange, new52WHL, prev52WHL, prevClose, prevHLDate, highLowPrice

6. Execution Guide & Testing

Run Core Ingestion (Produces 4 CSV Files)

```
python main.py
```

Launch the Interactive Web Dashboard

```
python main.py --ui
# Navigate to: http://127.0.0.1:8000/
```

Run the Automated Test Suite

```
pytest
# Result: 47 passed in 1.61s (100% coverage across client, validator, storage, pipeline)
```

> **Conclusion**
>
> The application meets all core engineering assignment specifications: automated retrieval of all 4 datasets without manual copying, clean date-stamped CSV filenames, fault-tolerant network retries, and comprehensive unit tests.